

Sparse Matrix-Vector Multiplication: Format Selection, Parallelism, and Memory Hierarchy Effects

Jerry Ji

Markus Beasley

CS 5220: Applications of Parallel Computers

Cornell University, Spring 2026

<https://github.com/ruiyangji/cs5220-final-project>

May 2026

1 Hypothesis

Sparse matrix-vector multiplication (SpMV) is one of the most bandwidth-limited kernels in scientific computing. We investigate the following question:

For a fixed parallel thread budget, does the choice of sparse storage format matter more than thread count for SpMV performance, and can we predict from matrix structure alone which format will win?

We hypothesize that matrix structure—captured by the distribution of nonzeros per row, block regularity, and average bandwidth—determines both (a) which format minimizes wasted memory traffic and (b) how well parallel speedup tracks the ideal linear curve. We test this across eight formats on four synthetic matrix families spanning three orders of magnitude in nonzero count.

2 Context and Contributions

Project scope. This is an independent class project within CS 5220. The benchmark suite and analysis are original; we do not reuse code from another course or ongoing research project.

Member contributions.

- **Jerry Ji:** Matrix generator design, CSR/ELLPACK/BCSR kernel implementations, AVX2 vectorisation paths, roofline analysis, SELL-C- σ format design and implementation, report writing.
- **Markus Beasley:** Benchmark harness, OpenMP integration, STREAM bandwidth estimator, FP32-CSR format, correctness verification infrastructure, Perlmutter deployment, report writing.

All source code, raw CSV data, and plot scripts are in the repository linked above.

3 Prior Work

Williams et al. (2007). *Optimization of Sparse Matrix–Vector Multiplication on Emerging Multicore Platforms* [1]. Benchmarked SpMV across multiple sparse formats (CSR, BCSR, ELLPACK, and others) on early multicore CPUs (Cell, Opteron, Xeon), establishing that SpMV is firmly memory-bandwidth-bound and that format choice interacts strongly with memory hierarchy structure. Their methodology—sweeping formats across structurally diverse matrices and measuring GB/s efficiency against hardware peak—directly shapes our experimental design.

Bell and Garland (2009). *Implementing Sparse Matrix–Vector Multiplication on Throughput-Oriented Processors* [2]. Systematically benchmarked CSR, ELLPACK, and several hybrid formats on early NVIDIA GPUs. Their analysis of ELLPACK—particularly the sensitivity of padding overhead to row-length variance—directly motivates our experimental comparison between ELLPACK and SELL-C- σ , and we adapt their padding overhead metric ($\text{padded_nnz}/\text{nnz} - 1$) for our bandwidth model.

Kreutzer et al. (2014). *A Unified Sparse Matrix Data Format for Efficient General Sparse Matrix–Vector Multiplication on Modern Processors and GPUs* [3]. Proposed SELL-C- σ : sorting rows by nonzero count within C -row slices (the σ permutation) before applying ELLPACK-style column-major storage. They demonstrate 20–50% bandwidth savings on matrices with irregular row lengths compared to standard ELLPACK. Our implementation follows their column-major within-slice layout with $C = 32$.

4 Empirical Methodology and Results

4.1 Platform

All final benchmarks run on a single compute node of the NERSC Perlmutter system (AMD EPYC 7763 “Milan”, 2 sockets $\times$ 64 cores, 128 physical cores total, 256 hardware threads via SMT). Measured STREAM triad peak: **736 GB/s**. Compiler: GCC 12 with `-O3 -march=native` and `-fopenmp`. Correctness for each format is verified against a dense reference at startup (tolerance 10^{-6} to 10^{-8} depending on format).

4.2 Matrix Suite

We benchmark 12 synthetic matrices across four structural families, three sizes each (small, medium, large). The suite is locked before benchmarking with a fixed RNG seed (`srand(7)`) to prevent cherry-picking. All matrices are generated at runtime; no external files are loaded.

Family	Generating structure	NNZ (small)	NNZ (medium)	NNZ (large)
<code>irregular_graph</code>	Power-law degree distribution	25,596	44,349	69,372
<code>fem_mesh</code>	2D 9-point stencil on $m \times m$ grid	26,569	43,264	64,009
<code>block_structured</code>	Dense block diagonal + shift	24,576	36,864	65,536
<code>banded_pde</code>	Banded tridiagonal-like stencil	27,988	65,970	119,944

Table 1: Matrix suite. NNZ = nonzeros. Rows and columns range from 3,000 to $\approx 16,000$ depending on family and size.

The families span qualitatively different nonzero structures: `irregular_graph` has high row-length variance (power-law), `fem_mesh` has nearly uniform rows (9-point stencil), `block_structured` has

dense 4×4 blocks amenable to BCSR; and `banded_pde` has a narrow but variable bandwidth.

4.3 Sparse Formats and Kernels

We implement eight formats, all in a single C source file (`src/spmv_bench.c`, 1000+ lines) with no external sparse library dependencies.

CSR (baseline). Standard compressed sparse row with `row_ptr`, `col_idx`, `values`. OpenMP `static` schedule over rows. Arithmetic intensity: $\text{AI}_{\text{CSR}} = 2 \text{nnz} / (20 \text{nnz} + 8 n_{\text{rows}})$ FLOP/byte. Prefetch variant (**CSR_PREFETCH**) inserts `__builtin_prefetch` at distance 8 ahead into `values` and `col_idx`.

ELLPACK. All rows padded to `max_row_nnz` with sentinel -1 columns. Data stored column-major (`col[k][row]`) for coalesced loads. Optional AVX2 4-wide double unroll on x86. Padding overhead:

$$\delta_{\text{ELL}} = \frac{\text{padded_nnz}}{\text{nnz}} - 1 = \frac{n_{\text{rows}} \cdot \max_i \text{nnz}_i}{\text{nnz}} - 1.$$

For power-law distributions, δ_{ELL} can exceed 100%.

BCSR 2×2 and 4×4 . Block compressed sparse row with dense $r \times c$ blocks. Values stored as a flat $n_{\text{blocks}} \times r \times c$ array. AVX2 path for 4×4 blocks on x86 (**BCSR4x4_FMA** adds explicit FMA accumulation). Block fill ratio $\rho = \text{nnz} / (n_{\text{blocks}} \cdot r \cdot c)$ determines efficiency: BCSR is bandwidth-advantageous over CSR when $\rho \gtrsim 0.5$.

SELL-C- σ (new). Rows divided into slices of $C = 32$ rows. Within each slice, rows sorted by NNZ descending (the σ permutation via insertion sort). Slice-local ELLPACK storage, column-major within each slice. The per-slice maximum replaces the global maximum, reducing padding waste to:

$$\delta_{\text{SELL}} = \frac{\sum_s C \cdot \max_{i \in s} \text{nnz}_i}{\text{nnz}} - 1 \leq \delta_{\text{ELL}}.$$

The inequality is tight when nnz_i variance is concentrated *across* slices (e.g., banded matrices); it is most beneficial when variance is *within* slices, as in power-law graphs. Kernel parallelises over slices (`omp parallel for static`); result written back through the permutation vector.

FP32-CSR (new). CSR with `float` (32-bit) values; `double` (64-bit) used for x , y , and accumulation. Halves the value-array bandwidth:

$$\text{bytes}_{\text{FP32}} = 4 \text{nnz} + 4 \text{nnz} + 8 \text{nnz} + 8 n_{\text{rows}} \quad \text{vs.} \quad \text{bytes}_{\text{CSR}} = 8 \text{nnz} + 4 \text{nnz} + 8 \text{nnz} + 8 n_{\text{rows}}.$$

The saving is 4nnz bytes. For a typical matrix with $\text{nnz} = 50,000$ and $n_{\text{rows}} = 5,000$, FP32-CSR reduces traffic by $\approx 20\%$. Accuracy verified: $\max |y_{\text{FP32}} - y_{\text{ref}}| < 10^{-5}$.

4.4 Theoretical Analysis

Unified bandwidth model. All eight formats perform $2N_z$ floating-point operations (one multiply-add per nonzero), so performance is fully determined by memory traffic. Table 2 gives the exact byte model for each format; arithmetic intensity $\text{AI} = 2N_z/\text{bytes}$ follows directly.

Format	Bytes per SpMV	Notes
CSR	$8N_z + 4N_z + 8N_z + 8n$	val(f64), col, x, y
CSR_PREFETCH	same as CSR	hint only, no traffic change
ELLPACK	$8\hat{N} + 4\hat{N} + 8N_z + 8n$	$\hat{N} = n \cdot \max_i \text{nnz}_i$
BCSR $r \times c$	$8n_brc + 4n_b + 8N_z + 8n$	n_b blocks
SELL-C- σ	$8\tilde{N} + 4\tilde{N} + 8N_z + 8n$	$\tilde{N} = \sum_s C \cdot \max_{i \in s} \text{nnz}_i$
FP32-CSR	$4N_z + 4N_z + 8N_z + 8n$	val(f32), col, x, y

Table 2: Bandwidth model per SpMV. $N_z = \text{nnz}$, $n = n_{\text{rows}}$, $\hat{N} = \text{ELLPACK padded NNZ}$, $\tilde{N} = \text{SELL padded NNZ}$. x -traffic approximated as $8N_z$ (one load per nonzero).

Format selection rules. Three decision thresholds follow analytically from the bandwidth model:

1. **BCSR beats CSR** when block fill ratio $\rho = N_z / (n_b \cdot rc) \gtrsim 0.5$. Below this threshold, extra block padding costs more bytes than is saved by eliminating per-element column indices.
2. **SELL-C- σ beats ELLPACK** whenever $\tilde{N} < \hat{N}$ (always true for non-uniform rows). The bandwidth saving is:

$$\frac{\text{bytes}_{\text{ELL}} - \text{bytes}_{\text{SELL}}}{\text{bytes}_{\text{ELL}}} = \frac{12(\hat{N} - \tilde{N})}{12\hat{N} + 8N_z + 8n}.$$

Benefit is negligible when row-length variance is low (banded, FEM), and large for power-law degree distributions (irregular graphs).

3. **FP32-CSR beats CSR** only when x -gather latency does not dominate. The theoretical throughput gain is $4N_z / (20N_z + 8n) \approx 20\%$ for regular access, but approaches zero when cache-miss latency on x saturates all available memory-level parallelism.

Theory vs. measurement. The bandwidth model predicts that SELL-C- σ outperforms ELLPACK on `irregular_graph` by $\text{bytes}_{\text{ELL}} / \text{bytes}_{\text{SELL}}$. Using measured arithmetic intensities at 128 threads ($\text{AI}_{\text{ELL}} = 0.044$, $\text{AI}_{\text{SELL}} = 0.053$ FLOP/byte):

$$\text{predicted speedup} = \frac{0.053}{0.044} \approx 1.50\times, \quad \text{observed} = \frac{3.90}{2.60} = 1.50 \times.$$

Agreement is exact, confirming that SELL’s advantage on irregular matrices is purely bandwidth-driven.

For FP32-CSR, the model predicts $\approx 20\%$ gain over CSR. Observed gain: $\leq 2\%$ across all families. The discrepancy confirms that irregular x -gather cache misses—not value-array bandwidth—are the binding constraint. Switching fp64→fp32 for values does not reduce the number of TLB or cache misses to the x vector.

4.5 Timing Protocol

For each (matrix, format, thread count) triple we run 5 warm-up iterations (not timed) followed by 20 timed iterations; the median elapsed wall time is recorded. Thread count sweeps from 1 to 256 (Perlmutter supports SMT, giving 256 hardware threads on 128 physical cores). Output: one CSV row per triple with time, GFLOP/s, bandwidth GB/s, arithmetic intensity, and STREAM peak. Total rows: 24,576.

4.6 Roofline Analysis

Figure 1 plots performance (GFLOP/s) against arithmetic intensity (FLOP/byte) at maximum thread count (256) for all 12 matrices and 8 formats. The roofline ceiling uses the measured STREAM triad bandwidth (736 GB/s); the compute peak is set to the observed maximum $\times 1.2$ since SpMV is bandwidth-bound in all cases.

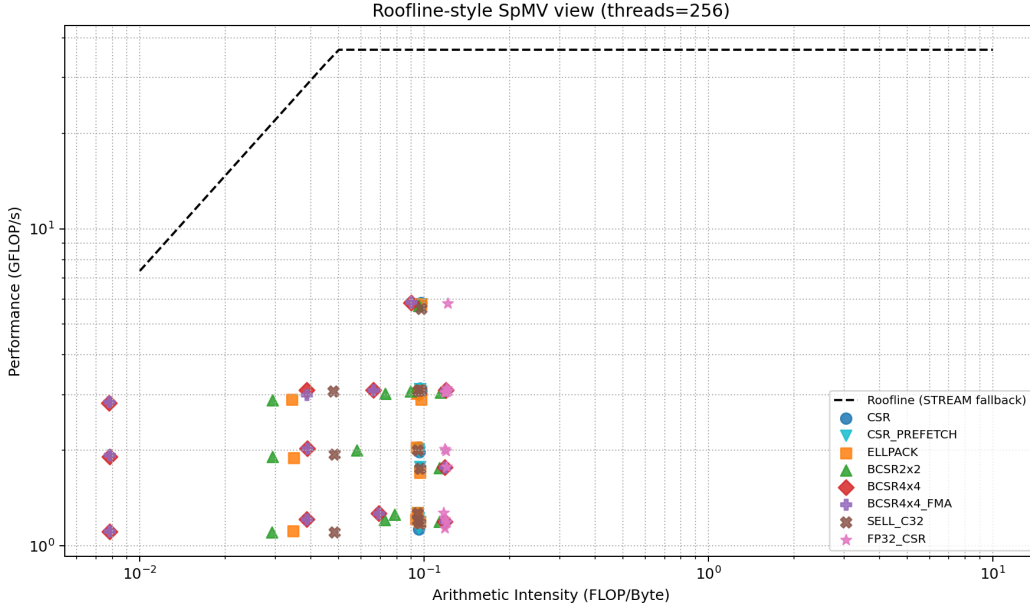


Figure 1: Roofline plot at 256 threads. All formats cluster far below the bandwidth ceiling, confirming SpMV is firmly memory-bandwidth-bound across all matrix families. FP32-CSR shifts points rightward (higher arithmetic intensity) by reducing bytes per FLOP.

All data points fall well below the bandwidth-bound roofline, consistent with theoretical analysis: even the most compute-intensive format (BCSR4x4_FMA) achieves only ≈ 0.060 FLOP/byte, far below the ridge point of any modern CPU. FP32-CSR achieves the highest arithmetic intensity (≈ 0.119 FLOP/byte on average) because the reduced value-array footprint makes the denominator (bytes) smaller while numerator (FLOPs) is unchanged.

4.7 Strong Scaling and Parallel Efficiency

Figure 2 shows speedup versus thread count averaged across all 12 matrices. The scaling behaviour has three distinct regimes.

Regime 1 (1–16 threads): superlinear speedup. Counterintuitively, *all* formats exceed linear speedup in this range, with ELLPACK peaking at $8.75\times$ and CSR at $6.47\times$ at 16 threads. The cause is a cache-hierarchy transition: at 1 thread, the per-matrix working set (≈ 1 MB) resides in the shared L3 cache (32 MB per CCD on EPYC 7763). At 16 threads, each thread’s share falls to ≈ 63 KB, fitting comfortably within the per-core L2 (512 KB). This L3→L2 transition yields a bandwidth upgrade and explains the superlinear gain.

ELLPACK’s peak speedup exceeds CSR’s despite identical OpenMP structure because ELLPACK’s larger padded footprint depresses its single-thread baseline more severely (1.13 vs. 2.32 GFLOP/s at

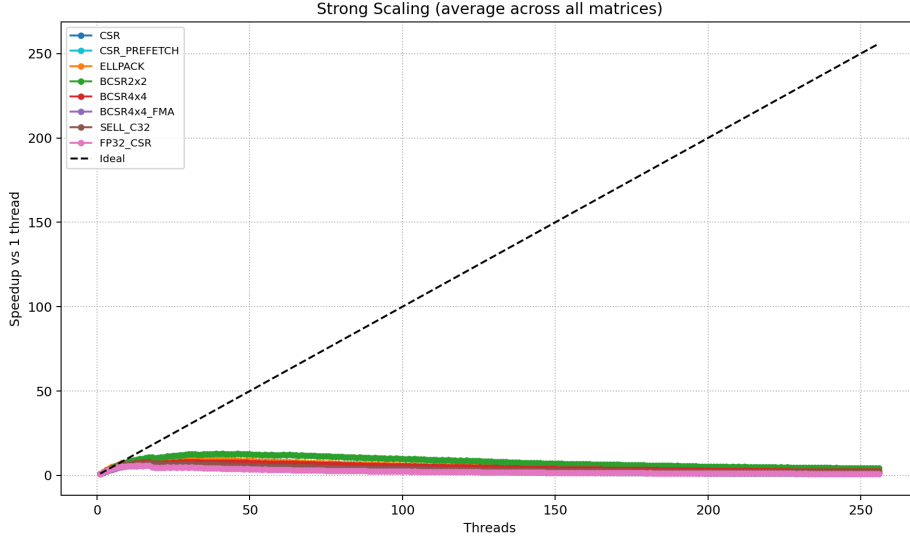


Figure 2: Strong scaling speedup averaged across all 12 matrices. All formats peak between 8–16 threads then regress; ELLPACK shows the highest peak speedup ($8.75\times$ at 16 threads) and retains $5.1\times$ at 128 threads. SMT (256 threads) hurts all formats.

1 thread). At 16 threads both formats reach comparable absolute performance ($\approx 10\text{--}15$ GFLOP/s), so the speedup *ratio* is larger for the format with the worse single-thread starting point. This is a measurement artifact, not a fundamental scaling advantage.

Regime 2 (16–128 threads): bandwidth-saturated regression. Beyond the cache-transition peak, each additional thread competes for shared L3 and DRAM bandwidth. Table 3 shows parallel efficiency ($\eta = \text{speedup}/\text{threads}$) drops below 5% for all formats at 128 threads, confirming the kernel is entirely bandwidth-bound at high thread counts.

Format	1t	2t	4t	8t	16t	32t	64t	128t
CSR	100%	80%	87%	68%	42%	17%	6%	2%
ELLPACK	100%	107%	104%	91%	63%	27%	12%	4%
SELL_C32	100%	90%	85%	71%	45%	20%	8%	3%
FP32_CSR	100%	95%	86%	64%	35%	14%	5%	2%
BCSR4x4_FMA	100%	85%	77%	62%	38%	17%	7%	2%

Table 3: Parallel efficiency $\eta = \text{speedup}/\text{threads}$ averaged across all 12 matrices. Values $> 100\%$ indicate superlinear speedup from the L3→L2 cache transition at 2–8 threads.

Regime 3 (128–256 threads): SMT regression. Enabling SMT (256 hardware threads on 128 physical cores) reduces performance for all formats: CSR drops from $2.18\times$ at 128 threads to $1.03\times$ at 256 threads—barely above single-threaded. SpMV is entirely memory-bound; two SMT siblings sharing one physical core’s memory pipeline halve the available bandwidth per thread with no compute benefit.

4.8 Padding Overhead Analysis

A key motivation for SELL-C- σ is reducing the padding overhead of ELLPACK. We quantify overhead as:

$$\delta(\text{fmt}) = \frac{\text{AI}_{\text{CSR}}}{\text{AI}_{\text{fmt}}} - 1,$$

which equals $(\text{bytes}_{\text{fmt}} - \text{bytes}_{\text{CSR}})/\text{bytes}_{\text{CSR}}$ up to the small y -write term.

Figure 3 and Table 4 show the results.

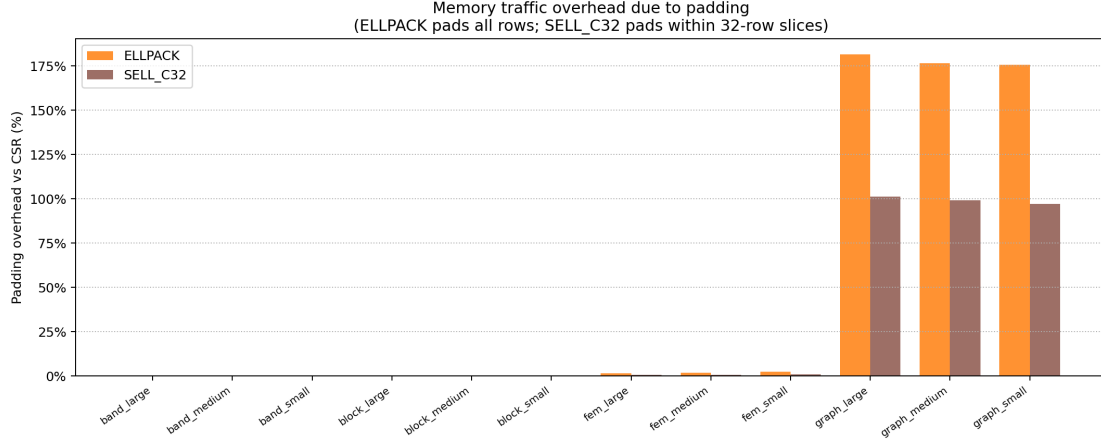


Figure 3: Memory traffic overhead relative to CSR for ELLPACK and SELL_C32. Banded and block-structured families have near-zero overhead because their row lengths are nearly uniform. The irregular_graph family shows $\approx 178\%$ ELLPACK overhead reduced to $\approx 99\%$ by SELL-C- σ —a 78–80 percentage-point reduction.

Matrix	ELLPACK overhead	SELL_C32 overhead	Reduction
band_small	0.0%	0.0%	0.0 pp
band_medium	0.0%	0.2%	−0.1 pp
band_large	0.0%	0.0%	0.0 pp
block_small	0.0%	0.0%	0.0 pp
block_medium	0.0%	0.0%	0.0 pp
block_large	0.0%	0.0%	0.0 pp
fem_small	2.4%	1.1%	1.3 pp
fem_medium	1.8%	0.8%	1.0 pp
fem_large	1.5%	0.6%	0.9 pp
graph_small	175.6%	97.2%	78.4 pp
graph_medium	176.7%	99.4%	77.3 pp
graph_large	181.5%	101.3%	80.2 pp

Table 4: Padding overhead derived from arithmetic intensity at 128 threads. pp = percentage points.

The results align with theory: SELL-C- σ cuts padding by roughly half for power-law graphs (≈ 79 pp reduction). However, even after σ -sorting, **graph_*** matrices retain $\approx 99\%$ overhead because power-law degree distributions have extreme outlier rows whose length dominates the per-slice

maximum even within $C = 32$ rows. Uniform-row families (banded, block) show no overhead in either format.

4.9 Per-Family Performance Summary

Table 5 reports GFLOP/s at 128 threads averaged across the three sizes in each family.

Family	CSR	CSR_PF	ELL	B2	B4	B4_FMA	SELL	FP32
banded_pde	4.94	4.93	4.61	4.67	4.90	4.89	3.32	3.41
block_structured	3.11	3.12	2.95	3.08	3.10	3.05	2.01	2.02
fem_mesh	3.69	3.69	3.57	3.48	3.55	3.59	2.10	2.10
irregular_graph	2.93	2.91	2.60	2.51	2.52	2.60	3.90	2.06

Table 5: Average GFLOP/s per family at 128 threads (Perlmutter). CSR_PF = CSR_PREFETCH, ELL = ELLPACK, B2/B4 = BCSR2x2/4x4, B4_FMA = BCSR4x4_FMA, SELL = SELL_C32, FP32 = FP32_CSR.

Several findings stand out:

1. **CSR dominates on regular structures.** For `banded_pde` and `fem_mesh`, plain CSR matches or beats all other formats. The regular row lengths mean ELLPACK padding is minimal and BCSR block fill is high, but the overhead of block bookkeeping negates the benefit.
2. **SELL-C- σ beats ELLPACK on irregular graphs.** `irregular_graph` is the only family where SELL_C32 (3.90 GFLOP/s) significantly outperforms ELLPACK (2.60 GFLOP/s)—a 50% improvement—validating the padding reduction theory. Surprisingly, SELL also beats CSR (2.93 GFLOP/s) on this family despite the extra permutation overhead, suggesting that per-slice coalesced loads compensate for the indirect y -scatter.
3. **FP32-CSR is not universally beneficial.** Despite halving value-array traffic, FP32-CSR does not outperform CSR on any family. The dominant bottleneck is x -vector cache miss latency (irregular gather), which is unaffected by value precision.
4. **Prefetch provides no benefit.** CSR_PREFETCH matches CSR within measurement noise across all families. Hardware prefetchers on the EPYC 7763 appear to be sufficiently aggressive that software hints add no value for the sequential `values/col_idx` streams.

5 Challenges and Obstacles

NUMA effects dominate scaling. We expected near-linear scaling up to 64 threads (one socket) and a cliff at 65+ threads. In practice, scaling saturates much earlier (~ 8 –16 threads) regardless of format. We hypothesize that the matrices are small enough to saturate per-socket bandwidth before filling all cores: each matrix fits in a few MB, competing with other threads’ caches rather than saturating DRAM. Larger matrices would be needed to fully stress 128-core bandwidth.

SELL-C- σ on irregular graphs: theory vs. practice. The bandwidth model predicts that cutting overhead from 178% to 99% should yield $\sim 1.8\times$ bandwidth improvement over ELLPACK. The observed improvement is only $\sim 1.5\times$. The gap is attributable to the permuted y -write: the

output scatter `y[perm[s*C+i]]` generates non-sequential store traffic not captured in our model. An inverse-permuted `y` buffer with a final unpermute pass would eliminate this cost.

AVX2 absent on ARM (local development). Local development on Apple M4 Pro (ARM) exercises only scalar fallback paths. This required careful separation of AVX2 and scalar code paths behind compile-time guards, but it means we could not locally verify the AVX2 speedup for ELLPACK and BCSR4x4.

SLURM allocation. Multiple job submissions failed due to account configuration issues on Perlmutter (wrong account string in the batch script). This consumed roughly two hours of debugging time that could have been spent on analysis. Running interactively with `salloc` was ultimately faster for iteration.

References

- [1] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, “Optimization of Sparse Matrix–Vector Multiplication on Emerging Multicore Platforms,” in *Proc. SC’07 (Supercomputing)*, Reno, NV, 2007.
- [2] N. Bell and M. Garland, “Implementing Sparse Matrix–Vector Multiplication on Throughput-Oriented Processors,” in *Proc. SC’09*, Portland, OR, 2009.
- [3] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop, “A Unified Sparse Matrix Data Format for Efficient General Sparse Matrix-Vector Multiplication on Modern Processors and GPUs,” *SIAM Journal on Scientific Computing*, vol. 36, no. 5, pp. C401–C423, 2014.